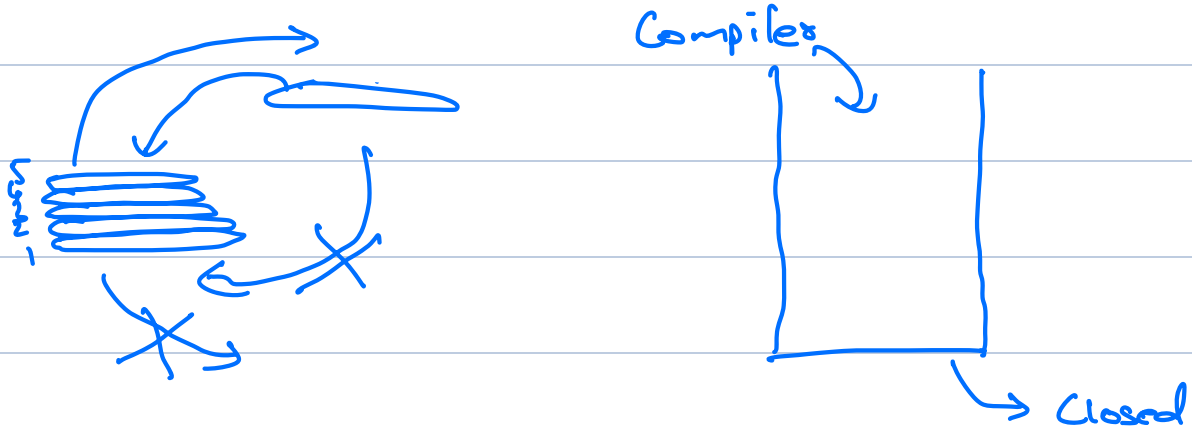
Stacks and Queues

Stack: Pile of items on top of each other.



Stacks are linear data structures to store information from bottom to top.

* The last item that got inserted gets removed first

→ This is LIFO (Last In First Out) mechanism.

Applications

- Function call tracing.
- Undo / Redo.

Operations

1) Push - Append to top.
2) Pop - Remove from top.
3) Peek - Look at the top.
4) isEmpty - Check if stack is empty.

} $O(1)$

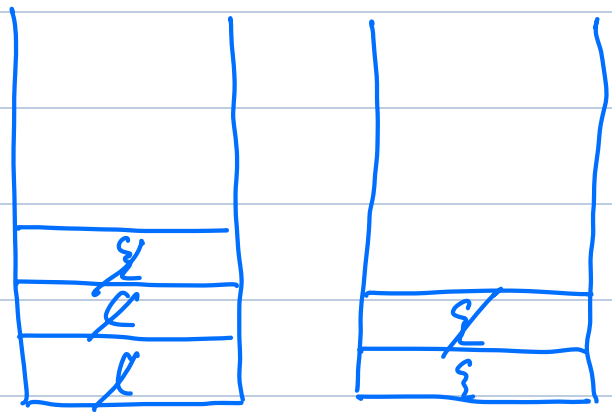
Question: Check whether the given sequence of paranthesis is valid.

ex: s = "(({}))"

O/P = True.

s = "{{}}"

O/P = False.



```
def checkValidParanthesis(str):
    s = []  # Empty Stack
    for char in str:
        # Opening paranthesis
        if (char == "{" or char == "(" or
            char == "[") :
            s.append(char)
        if (char == "}"):
            if (s and s[-1] == "{") :  s.pop()
            else:  return False
        if (char == ")"):
            if (s and s[-1] == "(") :  s.pop()
            else:  return False
```

```
    if (char == "]"):
        if (s and s[-1] == "[") :   s.pop()
        else :  return False
if (s) : return False.   # Check if empty.
return True.
```

str = ")(" str = "("

s = [] s = ["("]

Question: Given a string, remove equal pair of consecutive characters till possible.

ex: s = "abcddc"

O/P = "ab"

s = "abbcbbcacx"

O/P = "cx"

```
def removeRepeatingCharacters(str):
    s = []
    for char in str:
        if(s and s[-1] == char):
            s.pop()  # Removes matching pair
        else:
            s.append(char)
    return "".join(s)
```

Queue: Sequence of members/elements addressed from the beginning.

Insertion order - 1 2 3

Serving order - 1 2 3

* First incoming input gets served first.
This is called FIFO (First In First Out) Order.

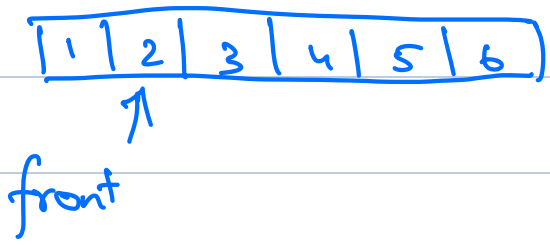
Applications

- Server load handling.
- Message queues.
- Ticket handling systems.

Operations

1) Enqueue - Adds items to the end
2) Dequeue - Removes items from the start.
3) Peek - Look at the front.
4) isEmpty - Checks if queue is empty.

$\Big\}\ O(1)$

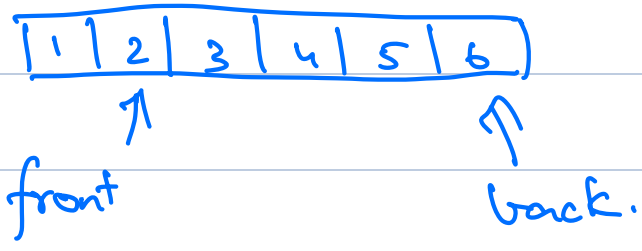


Dequeue just moves the "front" pointer by 1.

Doubly Ended Queue

Applications

- Youtube Videos.



Question: You're building a gatekeeper for requests. At any moment, only allow a request if the number of requests in the last T seconds is less than N. Otherwise, deny it.

ex: $N = 3$ $T = 10$

Req = [0, 1, 2, 9, 10] (Time of occurrence)

✓ ✓ ✓ ✗ ✓

O/P = 4.

* Servers can only handle x requests in a timeframe. So gatekeepers are built.

$N = 3$ $T = 10$

cur

[0, 1, 2, 8, 9, 10, 11, 12, 13, 15, 20]

✓ ✓ ✓ ✗ ✗ ✓ ✓ ✓ ✗ ✗ ✓

10 11 12 20

Gate

Server

		2	10	11

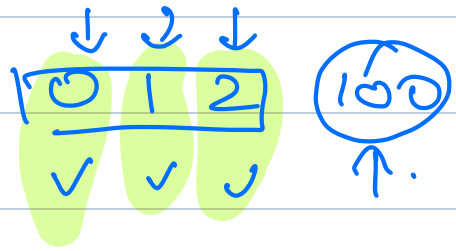
front

Capacity = 3 requests.

Processing time = 10 secs.

* You need a sliding window of t seconds.
* Number of requests in the sliding window are the number of requests being served.
* If there are N requests in the window, then deny the new request.
* After the T second window, deque the requests in the front.

```
def gateKeeper(req, N, t):
    q = []
    handled = 0
    for cur in req:
        # Dequeue the processed requests.
        while( q and cur - q[0] >= t):
            q.pop(0)
        if (len(q) < N):
            q.append(cur)
            handled += 1
    return handled.
```



cur

[0, 1, 2, 8, 9, 10, 11, 12, 13, 15, 20

q = [~~0~~, ~~1~~, ~~2~~, ~~10~~, 11, 12, 20 N = 3

handled = ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~ ~~6~~ 7 t = 10